Week 5 AS2: Class Activity and Reflection

Anthony Bayate | Kean University | Web Client-Side Programming | Neon Defense (tower defense)

Live game

https://ab-as2-neon-defense.vercel.app

What I built

Neon Defense is a tower-defense game written from scratch in plain HTML, CSS and JavaScript (index.html, style.css, game.js, no libraries). Enemies follow a winding neon path to the core. The player spends credits on four tower types (Pulse, Cannon with splash damage, Frost that slows, Sniper with long range), can upgrade each tower to level 3 or sell it, and has to survive 20 waves. Every fifth wave ends with a boss. The best wave reached is saved with localStorage. I avoided simple games on purpose: this one has pathing, targeting, wave scaling, an economy and upgrade/sell decisions.

How I used HTML, CSS and JS

- HTML: page structure, the shop, stats panel and start/game-over overlay.
- CSS: dark neon theme, grid layout for the shop and stats, hover/selected states, responsive canvas.
- JS: canvas rendering, a requestAnimationFrame game loop, enemy path-following, tower targeting (furthest enemy along the path), projectiles, splash and slow effects, a wave generator with HP scaling, DOM-based UI and keyboard shortcuts (1-4, Space, P, Esc).

How I built it with an LLM (step by step)

- Asked the LLM to walk me through the project: pick a non-trivial game, plan the three files, and describe the game loop before writing code.
- Built it in stages: map + path, enemies moving, towers shooting, then economy, waves, upgrades, polish.
- Ran it after each stage and fed problems back. Example: late waves were too hard, so I lowered the HP growth from 1.17x to 1.14x per wave.
- Ran a scripted playthrough in the browser console (places towers, fast-forwards waves) to check the whole loop for errors before deploying.

- Deployed with the Vercel CLI: install it, run "vercel login", then "vercel --prod" in the project folder. Vercel serves the static files directly, so no build step is needed.